
Desarrollo de un Robot Movil Autónomo

Programación de nodos sensores para IoT

Jaime Nogueira Fuentes



Universidad Complutense de Madrid

Facultad de Ciencias Físicas

Índice

1. Introducción	1
1.1. Contexto Tecnológico y Justificación	1
1.2. Objetivos del Proyecto	1
2. Estado del Arte	1
2.1. Análisis de la Arquitectura Monolítica (NVIDIA JetBot)	1
2.1.1. Capacidades de Procesamiento (Fortalezas)	2
2.1.2. Limitaciones Estructurales y Determinismo	2
3. Arquitectura del Sistema: La Plataforma Arduino UNO Q	3
3.1. Cambio de Paradigma: De Bare-Metal a Zephyr RTOS	3
3.1.1. Limitaciones del Super-Loop Tradicional	3
3.1.2. La Solución Zephyr OS	3
3.2. Interconexión e Interfaces	4
3.2.1. Mecanismo de Interconexión (Bridge)	4
3.2.2. Interfaces de Hardware Explotadas	4
4. Metodología: Arquitectura y Construcción de Drivers	4
4.1. Desarrollo del Hardware Abstraction Layer (Drivers)	4
4.1.1. Driver de Tracción y Cinemática (Motores DC)	4
4.1.2. Driver de Navegación Inercial (IMU MPU6050)	5
4.1.3. Driver de Sensores de Distancia (Sonar)	6
4.1.4. Driver de Gestión de Energía (BatteryMonitor)	7
4.1.5. Driver de Velocidad (Encoders)	8
4.1.6. Driver de Interfaz Humana (Receptor IR NEC)	9
4.2. Análisis Detallado de los Estados de Control	10
4.3. Arquitectura de Software Basada en Hilos (Zephyr RTOS)	11
4.3.1. Jerarquía de Hilos Implementada	11
4.3.2. Sincronización y Protección de Datos (Mutex)	12
4.4. Implementación de Alto Nivel (Subsistema Linux)	12
4.4.1. Orquestación y Concurrencia (<code>main.py</code>)	12
4.4.2. Pasarela IoT Desacoplada (<code>iot_client.py</code>)	13
5. Montaje Físico	14
5.1. Compatibilidad del Ecosistema y Shield de Expansión	14
5.2. Integración Híbrida e Instrumentación Sensorial	14
5.3. Arquitectura de Alimentación y Limitaciones del USB-C	15
6. Resultados y Validación Experimental	16
6.1. Análisis de Recursos Computacionales (CPU/RAM)	16
6.2. Latencia y Telemetría IoT	17
6.3. Pruebas de Campo: Visión y Evasión	18
7. Conclusiones y Trabajo Futuro	18
7.1. Conclusiones Generales	18
7.2. Trabajo Futuro y Escalabilidad	19
7.2.1. 1. Unificación del Sistema de Energía (Hardware)	19
7.2.2. 3. Fusión Sensorial Robusta (EKF)	19
7.2.3. 4. Optimización de la Electrónica de Potencia	19

1. Introducción

1.1. Contexto Tecnológico y Justificación

La robótica móvil actual enfrenta una dicotomía crítica: la necesidad de ejecutar algoritmos de Inteligencia Artificial de alto nivel frente a la obligación de garantizar un control determinista en tiempo real. Tradicionalmente, esto se resolvía mediante arquitecturas distribuidas complejas (ej. Raspberry Pi conectada a Arduino), lo que introducía latencia y fragilidad.

La aparición de hardware heterogéneo como la **Arduino UNO Q** marca un cambio de paradigma al integrar en un solo PCB un microprocesador (MPU) para Linux y un microcontrolador (MCU) para tiempo real. Este proyecto surge para validar experimentalmente si esta arquitectura unificada es capaz de reemplazar a los sistemas distribuidos, explotando la comunicación interna entre núcleos para combinar la potencia de cómputo de la visión artificial con la fiabilidad de seguridad requerida en los actuadores.

1.2. Objetivos del Proyecto

El propósito principal de este trabajo es demostrar la viabilidad de una arquitectura de software jerárquica sobre la plataforma UNO Q. Para ello, se han definido los siguientes objetivos:

1. **Desarrollo de un Prototipo Cognitivo Híbrido:** Diseñar y construir un robot móvil diferencial autónomo capaz de realizar seguimiento de personas, integrando sensores exteroceptivos (cámara, sonar) y propioceptivos (IMU, batería) en un chasis unificado.
2. **Validación de la Arquitectura de Seguridad:** Demostrar un sistema donde la gestión física tiene prioridad absoluta sobre la inteligencia artificial. Se busca probar que delegar el control de actuadores a un RTOS (Zephyr en el MCU) garantiza que el robot se detenga ante obstáculos o fallos críticos, independientemente de la latencia o congelamiento de los procesos en Linux (MPU).
3. **Implementación de Sincronización MPU-MCU:** Establecer una comunicación robusta mediante el mecanismo *Bridge* (RPC), permitiendo que un script de alto nivel en Python (detección facial) guíe al robot sin comprometer el determinismo del bucle de control ejecutado en C++.
4. **Sistema de Telemetría IoT:** Desplegar una infraestructura de monitorización en tiempo real desacoplada, utilizando un ESP32 auxiliar para transmitir variables críticas (velocidad, rumbo, voltaje) a un panel web, validando la plataforma para aplicaciones de robótica de borde (*Edge Robotics*).

2. Estado del Arte

Para situar este proyecto en el contexto de la robótica móvil con capacidades de Inteligencia Artificial, se ha tomado como referente principal el estándar actual en la industria educativa y de prototipado: el robot **NVIDIA JetBot** [1].

2.1. Análisis de la Arquitectura Monolítica (NVIDIA JetBot)

El JetBot es una plataforma de robótica avanzada basada en el ordenador embebido **NVIDIA Jetson Nano**. Este sistema opera bajo una distribución completa de Linux (Ubuntu) y utiliza Jupyter Notebooks como interfaz de programación principal, centralizando todas las tareas en un único procesador (SoC).

2.1.1. Capacidades de Procesamiento (Fortalezas)

La principal ventaja de esta arquitectura reside en su potencia de cálculo bruto. Gracias a su GPU dedicada (arquitectura Maxwell de 128 núcleos), el JetBot posee una capacidad de inferencia en el borde (*Edge AI*) muy superior a los microcontroladores tradicionales.

- Permite ejecutar redes neuronales profundas (DNN) para tareas complejas como la regresión de caminos, clasificación de imágenes y evasión de colisiones en tiempo real.
- Utiliza librerías optimizadas como **TensorRT**, logrando altas tasas de cuadros por segundo (FPS) en el procesamiento de visión.

2.1.2. Limitaciones Estructurales y Determinismo

A pesar de su potencia en IA, el JetBot presenta una limitación crítica desde el punto de vista de la ingeniería de control: su arquitectura monolítica basada en un Sistema Operativo de Propósito General (GPOS).

Al carecer de un microcontrolador (MCU) dedicado para el tiempo real, presenta los siguientes problemas de seguridad y control:

- **Control en Espacio de Usuario:** La gestión de los motores se realiza directamente desde el espacio de usuario de Linux (típicamente mediante scripts en Python) a través del bus I2C.
- **Falta de Determinismo:** Linux, al no ser un sistema operativo de tiempo real (RTOS) por defecto, utiliza un planificador (*scheduler*) que prioriza el rendimiento global sobre la latencia. Si la carga de la CPU o GPU aumenta debido al procesamiento de imágenes, el hilo encargado de actualizar la velocidad de los motores puede sufrir retrasos o inanición (*starvation*).
- **Riesgo de Seguridad:** En situaciones de alta carga computacional, esta arquitectura puede provocar que el robot mantenga el último comando de movimiento activo durante más tiempo del deseado (latencia o *jitter*), comprometiendo la capacidad de frenado inmediato ante un obstáculo.

Esta limitación del estándar actual justifica la propuesta de este proyecto con la **Arduino UNO Q**, que busca resolver este conflicto mediante una arquitectura heterogénea que separa físicamente la inteligencia (MPU) de la seguridad reactiva (MCU).

3. Arquitectura del Sistema: La Plataforma Arduino UNO Q

El núcleo computacional del robot es la **Arduino UNO Q**, una placa de desarrollo de nueva generación que materializa la fusión tecnológica entre la facilidad de uso del ecosistema Arduino y la potencia de procesamiento industrial. A diferencia de los microcontroladores clásicos o los SBCs (Single Board Computers) tradicionales, la UNO Q implementa una arquitectura heterogénea en un único PCB. [2]

Esta plataforma se compone de dos subsistemas de cómputo independientes pero interconectados:

1. **El Procesador de Aplicaciones (MPU):** Un SoC **Qualcomm Dragonwing QRB2210**. Se trata de un procesador Quad-core Arm Cortex-A53 a 2.0 GHz que ejecuta un sistema operativo completo **Debian Linux**. Este subsistema incluye una GPU Adreno 702 y una NPU dedicada, encargándose de las tareas de alto nivel: gestión del stack de visión artificial, ejecución de modelos de IA y conectividad Wi-Fi 5 dual-band. En este proyecto, actúa como el Cerebro cognitivo que procesa las imágenes y gestiona las conexiones IoT.
2. **El Microcontrolador de Tiempo Real (MCU):** Un **STMicroelectronics STM32U585**. Basado en un núcleo Arm Cortex-M33 a 160 MHz, es el responsable del control de bajo nivel. Su función es garantizar el determinismo temporal necesario para el control de motores, la lectura de encoders y la gestión de sensores de seguridad (Ultrasonidos e IMU). Opera con lógica de 3.3V, manteniendo compatibilidad eléctrica con el ecosistema de sensores estándar.

3.1. Cambio de Paradigma: De Bare-Metal a Zephyr RTOS

Una de las innovaciones más disruptivas de la UNO Q, y que ha sido fundamental para el desarrollo de este robot, es el abandono del modelo de ejecución "Bare-Metal" tradicional de Arduino en favor de un Sistema Operativo de Tiempo Real (RTOS). [3]

3.1.1. Limitaciones del Super-Loop Tradicional

En las placas Arduino clásicas (como la UNO R3), el código se ejecuta en un bucle infinito secuencial (`loop()`). Este modelo, conocido como *Bare-Metal*, presenta graves limitaciones para robótica compleja: si el procesador se bloquea esperando una lectura de sensor (como un pulso ultrasónico), todo el control de motores se detiene, provocando movimientos erráticos.

3.1.2. La Solución Zephyr OS

En la UNO Q, el núcleo Arduino Core se ejecuta como una aplicación sobre el kernel de **Zephyr OS**. Esto transforma radicalmente la forma de programar el firmware:

- **Multitarea Preventiva (Preemptive Multitasking):** Permite dividir el código en hilos independientes (*Threads*). En este proyecto, se han separado el cálculo de odometría, la lógica de evasión y la telemetría en hilos distintos.
- **Determinismo Estricto:** Zephyr garantiza que las tareas de alta prioridad (como estabilizar el giroscopio) interrumpan siempre a las tareas de baja prioridad, asegurando la estabilidad física del robot. [3]

3.2. Interconexión e Interfaces

3.2.1. Mecanismo de Interconexión (Bridge)

La comunicación entre el entorno Linux (MPU) y el entorno de tiempo real (MCU) se realiza a través de **Arduino Bridge**, una librería RPC (Remote Procedure Call) basada en memoria compartida y transportes internos de alta velocidad. Esto elimina la necesidad de buses externos (UART/USB) propensos a fallos y latencia, permitiendo que el script de Python en Linux invoque funciones C++ en el microcontrolador de forma casi instantánea.

3.2.2. Interfaces de Hardware Explotadas

Para la construcción del robot, la placa ofrece interfaces críticas que han facilitado el diseño electrónico [4]:

- **Gestión de Energía Dual (PMIC):** La placa integra un PMIC avanzado que permite la alimentación híbrida. Se utiliza el pin VIN (7-24V) para la etapa de potencia de los motores, mientras que la lógica interna regula estos voltajes para alimentar el procesador Qualcomm y el STM32 sin necesidad de convertidores externos.
- **Conector USB-C Host/Device:** Permite la conexión directa de la cámara web al procesador de aplicaciones (Linux) para la captura de video de alta velocidad, aunque requiere una inyección de energía externa debido a las limitaciones de corriente del modo Host de la placa.

4. Metodología: Arquitectura y Construcción de Drivers

Esta sección detalla el núcleo de ingeniería del proyecto. Se ha optado por un enfoque de diseño “Bottom-Up”, comenzando por los controladores de hardware de bajo nivel y ascendiendo hasta la lógica de aplicación.

4.1. Desarrollo del Hardware Abstraction Layer (Drivers)

Uno de los mayores retos fue escribir los drivers para el STM32U585 bajo el ecosistema Zephyr, gestionando la concurrencia y los recursos compartidos. A continuación se detalla la construcción de cada driver.

4.1.1. Driver de Tracción y Cinemática (Motores DC)

El control de los motores se realiza mediante un puente H doble (L298N) gestionado por el microcontrolador. Durante las pruebas, se confirmó que el control directo mediante PWM presenta problemas físicos en motores económicos debido a la fricción interna. [5]

Problemática de la Zona Muerta Los motores DC presentan una “zona muerta” de voltaje no lineal. Si se aplica un ciclo de trabajo PWM bajo (por ejemplo, inferior a un umbral MIN_POWER), el motor consume corriente y emite zumbido, pero no genera par suficiente para vencer la fricción estática. Esto provoca que el robot no responda a comandos de velocidad baja, afectando a la precisión del seguimiento.

Implementación del Driver Para solucionar esto, se implementó una capa de abstracción en el archivo `motors.cpp` mediante la función `motors.drive`, la cual se apoya en una función de corrección llamada `clip.speed`. El driver realiza dos operaciones críticas:

1. **Gestión de Zona Muerta y Saturación:** La función `clip_speed` evalúa la velocidad solicitada. Si esta es distinta de cero pero inferior al umbral mínimo mecánico (`MIN_POWER`), el driver eleva automáticamente el voltaje al mínimo necesario para garantizar movimiento físico. Asimismo, satura los valores al rango máximo de 8 bits (± 255).
2. **Abstracción de Polaridad:** La función `motors_drive` traduce una velocidad con signo (ej. -150) a las señales lógicas necesarias para el puente H (pines IN1/IN2 para dirección y pin ENA para magnitud PWM), simplificando la lógica de control superior.

```

1 // Se limita la velocidad a un rango seguro y util
2 int clip_speed(int speed) {
3     if (speed == 0) return 0;
4     // Si es muy poco voltaje para moverlo, forzamos el minimo (MIN_POWER)
5     if (abs(speed) < MIN_POWER) return (speed > 0) ? MIN_POWER : -MIN_POWER;
6     // Saturacion maxima (8 bits)
7     if (speed > 255) return 255;
8     if (speed < -255) return -255;
9     return speed;
10 }
11
12 // Abstraccion de hardware: Convierte signo en direccion digital
13 void motors_drive(int speedLeft, int speedRight) {
14     speedLeft = clip_speed(speedLeft);
15     speedRight = clip_speed(speedRight);
16
17     // Ejemplo de control Motor A
18     if (speedLeft > 0) {
19         digitalWrite(IN1, HIGH); digitalWrite(IN2, LOW);
20         analogWrite(ENA, speedLeft);
21     } else if (speedLeft < 0) {
22         digitalWrite(IN1, LOW); digitalWrite(IN2, HIGH);
23         analogWrite(ENA, -speedLeft);
24     } else {
25         // Frenado
26         digitalWrite(IN1, LOW); digitalWrite(IN2, LOW);
27         analogWrite(ENA, 0);
28     }
29     // ... (Logica similar para Motor B)
30 }

```

Listing 1: Implementación real del driver de motores con gestión de zona muerta

4.1.2. Driver de Navegación Inercial (IMU MPU6050)

Para la estimación de la orientación, se desarrolló un driver en C++ (`MPU6050_Driver`) que abstrae la complejidad de los registros del sensor MPU6050. A diferencia de librerías genéricas bloqueantes, este controlador se diseñó para ser ligero y eficiente. [6]

Abstracción de Hardware y Lectura en Ráfaga El driver implementa el protocolo I2C utilizando la librería `Wire`. Para minimizar el tiempo de ocupación del bus, se optó por una estrategia de “Lectura en Ráfaga” (*Burst Read*). La función `readRawData` solicita 14 bytes consecutivos en una sola transacción I2C (Acelerómetro, Temperatura y Giroscopio), reduciendo drásticamente el *overhead* de comunicación comparado con leer registro a registro. Posteriormente, la función `getProcessedData` convierte estos valores crudos de 16 bits en unidades físicas (grados por segundo) aplicando el factor de escala configurado (por defecto $\pm 500^\circ/s$), entregando datos listos para la integración numérica.

Integración en Tiempo Real (RTOS) Dado que la integración de la velocidad angular ($\theta = \int \omega \cdot dt$) requiere un intervalo de tiempo (dt) constante para evitar la deriva, el driver se

invoca desde un hilo de alta prioridad en Zephyr.

- **Determinismo:** El hilo garantiza un muestreo exacto a 100Hz mediante `k_msleep(10)`.
- **Consistencia de Datos:** Aunque el driver maneja la comunicación I2C, la protección de los datos resultantes se delega al sistema operativo mediante Mutexes, asegurando que la lógica de navegación siempre acceda a un estado coherente del sensor.

```

1 void MPU6050_Driver::readRawData(...) {
2     // Lectura optimizada: 14 bytes de una sola vez
3     // (Acc-X/Y/Z, Temp, Gyro-X/Y/Z)
4     uint8_t buffer[14];
5     readBytes(MPU6050_REG_ACCEL_XOUT_H, 14, buffer);
6
7     // Reconstruccion de enteros de 16 bits (MSB | LSB)
8     ax = (int16_t)((buffer[0] << 8) | buffer[1]);
9     // ... (resto de ejes)
10    gz = (int16_t)((buffer[12] << 8) | buffer[13]);
11 }

```

Listing 2: Método de lectura eficiente en el driver IMU

4.1.3. Driver de Sensores de Distancia (Sonar)

Para la detección de obstáculos se implementó un controlador de bajo nivel para el sensor HC-SR04 (`sonar.cpp`). A diferencia de las implementaciones estándar, este driver evita el uso de funciones bloqueantes por defecto. [7]

Implementación de Lectura de Pulso (`my_pulseIn`) Durante el desarrollo se detectó que la función estándar `pulseIn()` de Arduino no estaba disponible en el núcleo basado en Zephyr de la placa UNO Q. Para resolver esta incompatibilidad, se implementó manualmente la función auxiliar `my_pulseIn`.

Esta función reconstruye el comportamiento de medición de ancho de pulso mediante el sondeo directo (*polling*) de los pines digitales utilizando `micros()`. Además, se aprovechó esta implementación propia para imponer un *timeout* de seguridad estricto de 30ms, garantizando que el sistema no quede bloqueado si el sensor ultrasónico falla o se desconecta.

Lógica de Disparo y Filtrado La función `sonar_get_distance` gestiona la secuencia de disparo (Trigger de $10\mu s$) y la conversión de tiempo a distancia ($v = 0.034cm/\mu s$). El driver aplica un filtro de rango físico:

```

1 // Leer Echo con timeout de 30ms (aprox 5 metros)
2 long duration = my_pulseIn(_echoPin, HIGH, 30000);
3
4 // Calculo en cm
5 int distance = (int)(duration * 0.034 / 2);
6
7 // Filtrado de ruido e infinito
8 // Si es mayor de 4m o 0 (timeout), se devuelve 0 indicando "fuera de rango"
9 if (distance > 400 || distance == 0) return 0;
10 return distance;

```

Listing 3: Lógica de filtrado en el driver del sonar

El valor de retorno 0 es interpretado por la lógica superior como “espacio libre” o “fuera de alcance”, evitando falsos positivos de colisión a larga distancia.

4.1.4. Driver de Gestión de Energía (BatteryMonitor)

El sistema de alimentación es crítico para la estabilidad de la placa Arduino UNO Q, que requiere un voltaje de entrada mínimo de 7.0V en el pin VIN. Para gestionar esto, se desarrolló una solución en dos fases: una caracterización offline mediante Python y una implementación en tiempo real en C++.

Fase 1: Caracterización y Modelado (Python) Antes de escribir el firmware, se sometió al paquete de baterías a un ciclo de descarga completo bajo carga real, registrando voltaje y consumo acumulado (mAh) mediante sensores INA219 y ACS715 en un archivo CSV.[8], [9]

Se desarrolló un script de análisis en **Python (Pandas/Matplotlib)** para procesar estos datos crudos y generar el modelo de descarga:

1. **Filtrado de Señal:** Se aplicó una media móvil (*rolling window*) de 50 muestras para eliminar el ruido eléctrico provocado por la conmutación de los motores.
2. **Definición de Zona Segura:** El script identifica matemáticamente el punto exacto de la curva donde el voltaje cae por debajo de 7.0V. Todos los datos inferiores a este umbral son descartados, redefiniendo el 0 % lógico del robot en el límite de seguridad del hardware.
3. **Generación de LUT:** Finalmente, el script exporta automáticamente un array de C++ (*tablaVoltajes*) que mapea 21 puntos porcentuales (del 100 % al 0 %) a sus voltajes correspondientes en la curva real.

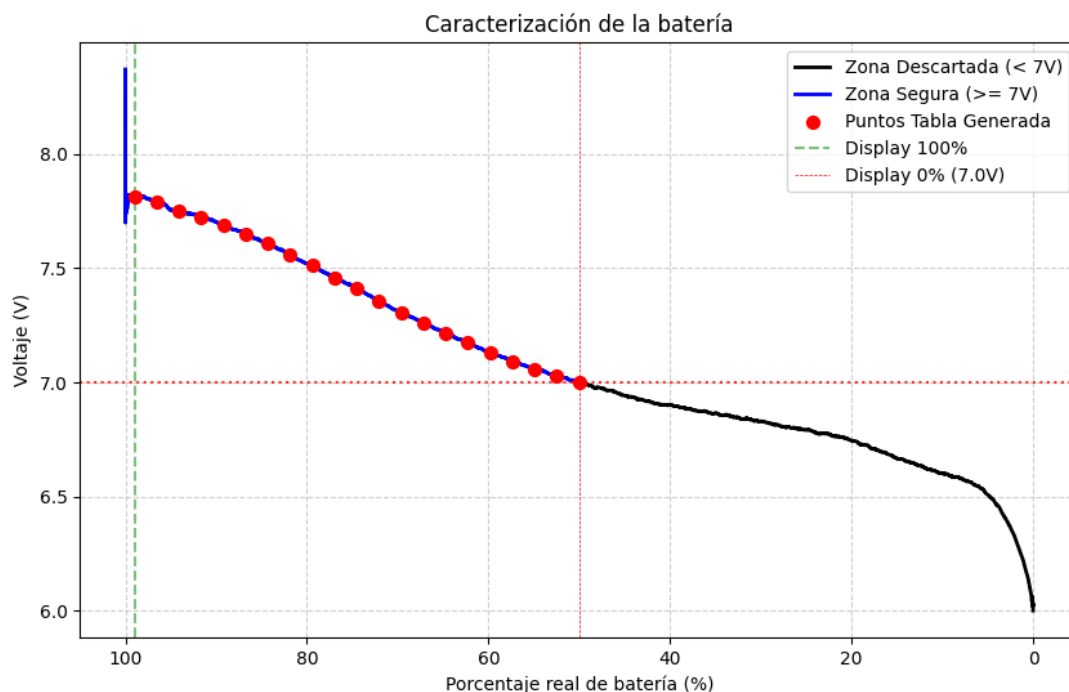


Figura 1: Curva de descarga caracterizada: Zona segura (Azul) vs Zona de corte (Negro)

Fase 2: Implementación del Driver (C++) El driver *BatteryMonitor* utiliza la tabla generada para ofrecer una estimación precisa sin carga computacional elevada.

- **Estimación Híbrida:** Al arranque, el sistema mide el voltaje en reposo e interpola su posición en la tabla (*Look-Up Table*) para estimar la carga inicial.

```

1 // Configuración: 0% Display = 7.0V (Corte estricto generado por script)
2 const float BatteryMonitor::VOLTAGE_TABLE[] = {
3     7.81, 7.79, 7.75, 7.72, 7.69, 7.65, 7.61, 7.56, 7.51,
4     7.46, 7.41, 7.36, 7.31, 7.26, 7.21, 7.18, 7.13, 7.09,
5     7.06, 7.03, 7.00
6 };

```

Listing 4: Tabla de calibración generada por Python

- **Ejecución (Coulomb Counting):** Durante el funcionamiento, para evitar fluctuaciones por caídas de tensión instantáneas (al acelerar motores), el driver integra la corriente consumida leída por el sensor:

$$Q(t) = Q_{inicial} - \int_0^t I(t) \cdot dt \quad (1)$$

Esto proporciona una lectura de batería estable y monótona decreciente, esencial para una telemetría fiable.

4.1.5. Driver de Velocidad (Encoders)

Para medir la velocidad lineal del robot y permitir la navegación por estima, se implementó el driver **encoders**. Dado que la lectura de pulsos mediante sondeo (*polling*) consumiría demasiados ciclos de CPU y perdería cuentas a altas velocidades, el diseño se basa estrictamente en interrupciones hardware.^[10]

Gestión de Interrupciones en C++ Un desafío técnico al trabajar con Orientación a Objetos en microcontroladores es que las Rutinas de Servicio de Interrupción (ISR) deben ser funciones estáticas o globales, lo que impide acceder directamente a las variables de una instancia de clase (como `count`). Para resolver esto, el driver implementa un patrón de “Trampolín Estático”: se define un array de punteros estáticos `instances`^[2] que almacena la referencia al objeto que gestiona cada pin. Cuando salta la interrupción física (ISR global), esta redirige la llamada a la instancia correcta, incrementando el contador de forma transparente.

Cálculo Diferencial de Velocidad La función `getSpeed` calcula la velocidad instantánea aplicando la definición de derivada discreta ($v = \Delta x / \Delta t$).

- Se calcula la distancia recorrida desde la última lectura: $\Delta x = (counts_{now} - counts_{last}) \cdot mPerPulse$.
- Se mide el diferencial de tiempo Δt usando `millis()`.
- Finalmente, se aplica un factor de conversión para entregar la telemetría directamente en km/h ($v \cdot 3.6$), facilitando la interpretación humana en el Dashboard.

```

1 // Interrupcion Global (Trampolin)
2 void SimpleEncoder::isr0() {
3     if (instances[0] != nullptr) {
4         instances[0]->count++; // Redireccion a la instancia
5     }
6 }
7
8 // Calculo cinematico
9 float SimpleEncoder::getSpeed() {
10     // Delta de tiempo y posicion
11     long dCount = c - lastCount;
12     float dt = (now - lastTime) / 1000.0;
13
14     // Velocidad (m/s) -> km/h

```

```

15     float speed = (dCount * mPerPulse) / dt;
16     return speed * 3.6;
17 }

```

Listing 5: Cálculo de velocidad y gestión de ISR estática

4.1.6. Driver de Interfaz Humana (Receptor IR NEC)

Para permitir el control manual y el cambio de modos sin depender de la conexión Wi-Fi, se integró un receptor infrarrojo. Debido a incompatibilidades de las librerías estándar de Arduino con el kernel de Zephyr en la gestión de timers, se desarrolló un decodificador para el protocolo NEC. [11]

Decodificación Asíncrona por Interrupciones El driver prescinde del sondeo (*polling*) para no desperdiciar ciclos de CPU. En su lugar, configura una interrupción externa (`attachInterrupt`) en el pin del receptor que se dispara en cada flanco de bajada (`FALLING`). Dentro de la Rutina de Servicio de Interrupción (ISR) `_ir_isr`, se mide el tiempo transcurrido desde el último flanco usando `micros()`. Esta diferencia temporal (*delta*) determina el significado del pulso según el estándar NEC:

- **Bit de Inicio:** Si el pulso dura más de $10ms$ ($13.5ms$ nominales), se reinicia la máquina de estados.
- **Lógica Binaria:** Para los bits de datos, se discrimina entre '0' y '1' por la duración del espacio entre pulsos. Si el tiempo es superior a $1600\mu s$, se interpreta como un '1' lógico; de lo contrario, es un '0'.

Concurrencia y Atomicidad Dado que la variable `_ir_code` (32 bits) se escribe dentro de una interrupción y se lee desde el bucle principal, existe riesgo de condición de carrera (lectura de un valor a medio escribir). Para evitarlo, se emplearon variables `volatile` y se implementó una sección crítica en la función de lectura:

```

1 // ISR: Maquina de estados basada en tiempo
2 void _ir_isr() {
3     unsigned long duration = current_micros - _last_micros;
4
5     // Deteccion de Start Bit (NEC > 10ms)
6     if (duration > 10000) {
7         bit_count = 0; temp_code = 0;
8     }
9     // Decodificacion de bits (Umbral 1600us para '1')
10    else {
11        temp_code <<= 1;
12        if (duration > 1600) temp_code |= 1;
13        // ... Si llega a 32 bits, bandera _ir_ready = true
14    }
15 }
16
17 // Lectura Atomica en el bucle principal
18 uint32_t remote_read() {
19     if (_ir_ready) {
20         noInterrupts(); // Inicio Seccion Critica
21         uint32_t code = _ir_code;
22         _ir_ready = false;
23         interrupts(); // Fin Seccion Critica
24         return code;
25     }
26     return 0;

```

Listing 6: Decodificación NEC y protección de sección crítica

4.2. Análisis Detallado de los Estados de Control

La Máquina de Estados Finita (FSM) gobierna el comportamiento del robot. A continuación, se detalla la lógica algorítmica implementada para cada caso, con especial énfasis en los modos autónomos complejos.

1. Modo de Navegación Inercial (STATE_FORWARD)

Es el estado por defecto cuando el robot avanza. En lugar de aplicar una potencia fija a los motores (lo que provocaría desviaciones por diferencias mecánicas), se utiliza un lazo de control cerrado con el giroscopio.

- **Control P (Proporcional):** El sistema calcula el error de rumbo ($error = \theta_{target} - \theta_{actual}$) y aplica una corrección diferencial a las ruedas:

$$V_{izq/der} = V_{base} \mp (K_p \cdot error)$$

Esto garantiza que el robot mantenga una línea recta perfecta incluso si una rueda tiene más fricción que la otra.

- **Disparo de Evasión:** En cada ciclo, se consulta el sonar. Si la distancia detectada es $0 < d < 20cm$ (umbral MIN_DISTANCE), la FSM fuerza una transición inmediata al estado de evasión.

2. Modo de Evasión Inteligente (STATE_AVOIDING_SEQUENCE)

A diferencia de los algoritmos reactivos simples que solo giran al chocar, este estado implementa una rutina de planificación local bloqueante (*Stop-Look-Decide-Act*). La secuencia, orquestada por la función `run_avoidance_sequence`, sigue estos pasos estrictos:

1. **Frenado y Estabilización:** Se detienen los motores y se espera 200ms para disipar la inercia y evitar que las vibraciones afecten a la lectura del sonar.
2. **Escaneo Activo:**
 - El servo gira la cabeza a 160° (Izquierda), espera 500ms y toma una muestra de distancia (D_{izq}).
 - El servo gira a 20° (Derecha), espera 1000ms y toma otra muestra (D_{der}).
 - Finalmente, recentra la cabeza a 90° .
3. **Árbol de Decisión:** El algoritmo compara ambas lecturas con un margen de seguridad ($OK_DISTANCE = 35cm$):
 - **Caso A** ($D_{izq} > D_{der}$ y $D_{izq} > 35cm$): El camino está libre a la izquierda. El robot aplica un giro pivotante sobre su eje (motores a $-220/220$) durante 1000ms.
 - **Caso B** ($D_{der} > D_{izq}$ y $D_{der} > 35cm$): El camino está libre a la derecha. Giro pivotante inverso.
 - **Caso C (Callejón sin salida):** Si ambas direcciones están bloqueadas, el robot retrocede durante 1 segundo y luego realiza un giro de 180° (2 segundos de rotación) para salir de la trampa.

4. **Reinicio:** Al terminar la maniobra, se actualiza el *Target Heading* del giroscopio al nuevo frente y se devuelve el control al estado FORWARD.

3. Modo de Seguimiento Visual (STATE_FOLLOWING_OBJECT)

Este estado acopla la visión artificial (Linux) con el control de tiempo real (Zephyr). Para resolver el problema de la latencia y la pérdida de objetivos, se implementó un control híbrido con tres capas de seguridad:

1. **Watchdog de Seguridad (Timeout):** Cada vez que el hilo lee datos de visión, verifica la marca de tiempo (`timestamp`). Si han pasado más de 1.5 segundos desde la última detección válida (por ejemplo, si el sujeto sale del encuadre o el script de Python falla), los motores se apagan automáticamente.
2. **Zona Muerta y Filtrado:** Se calcula el error horizontal del centroide ($E_x = X_{obj} - 160$). Si el error es menor a 20 píxeles, se considera despreciable ($E_x = 0$) para evitar oscilaciones nerviosas alrededor del centro.
3. **Control Dual (Histéresis):** El driver implementa dos estrategias de movimiento según la magnitud del error:
 - **Estrategia Tanque** ($|E_x| > 100px$): Si el objetivo está en los bordes de la imagen, el robot se detiene y gira sobre su propio eje a alta velocidad. Esto permite reorientar la cámara rápidamente sin desplazarse.
 - **Estrategia Suave** ($|E_x| \leq 100px$): Se aplica un control proporcional a la velocidad de las ruedas ($V = V_{base} \pm K_p \cdot E_x$). Aquí entra en juego la función `fixSpeed`, que garantiza que el voltaje nunca caiga en la "zona muerta" de los motores (por debajo de PWM 130), asegurando un movimiento fluido incluso con correcciones pequeñas.

4. Modos Manuales

Finalmente, los estados STATE_TURNING y STATE_BACKWARD responden directamente a los comandos del control remoto IR, anulando cualquier autonomía y permitiendo al operador recuperar el robot en situaciones de emergencia.

4.3. Arquitectura de Software Basada en Hilos (Zephyr RTOS)

Para superar las limitaciones del "Super-Loop" secuencial, el firmware se ha reescrito aprovechando el kernel de tiempo real de Zephyr OS. El sistema se descompone en tres hilos de ejecución (*Threads*) concurrentes, definidos mediante la macro `K_THREAD_STACK_DEFINE` y lanzados en el `setup` con prioridades estrictas.

Esta estructura garantiza el determinismo temporal: el planificador (*scheduler*) del RTOS asegura que el hilo de control inercial siempre tenga preferencia sobre la lógica de navegación o la telemetría.

4.3.1. Jerarquía de Hilos Implementada

1. **Hilo de Sensórica Inercial (Prioridad Alta - 2):** *Función:* `imu_thread_entry`
Este es el proceso más crítico del robot. Se ejecuta cada 10ms de forma estricta. Su función es leer el giroscopio MPU6050 e integrar numéricamente la velocidad angular (Z) para calcular el rumbo actual.
 - **Justificación:** Al asignarle la prioridad más alta (2), se evita que el sistema pierda muestras del sensor incluso si la visión artificial está saturando la CPU.



- *Recurso:* Pila dedicada de 4096 bytes.

2. **Hilo de Lógica y Control (Prioridad Media - 5):** *Función:* `logic_thread_entry`
Actúa como el cerebro del robot, ejecutando la Máquina de Estados Finita (FSM). Este hilo consume los datos de los sensores y toma decisiones de alto nivel (avanzar, girar, evadir).

- **Responsabilidades:**

- Gestionar la transición entre estados (`STATE_FOLLOWING`, `STATE_AVOIDING`, etc.).
- Leer el receptor IR para comandos manuales.
- Calcular los lazos de control proporcional para los motores basándose en el error de visión o de rumbo.

- *Recurso:* Pila ampliada de 8192 bytes para manejar la lógica compleja.

3. **Hilo de Telemetría (Prioridad Baja - 8):** *Función:* `telemetry_thread_entry`
Tarea de fondo que se ejecuta cada 500ms. Se encarga de recopilar el estado del sistema (voltaje de batería filtrado, velocidad de encoders y rumbo) y empaquetarlo en formato JSON para su envío a la interfaz web.

- **Justificación:** Al tener la prioridad más baja (8), si el robot necesita procesar una evasión de emergencia, la telemetría se pausará automáticamente, garantizando que el envío de datos nunca comprometa la seguridad física.

4.3.2. Sincronización y Protección de Datos (Mutex)

Dado que los datos de visión llegan de forma asíncrona desde Linux (vía RPC) y el IMU actualiza el rumbo en su propio hilo, existe riesgo de corrupción de memoria por acceso concurrente. Para solucionarlo, se implementó un cerrojo de exclusión mutua (`K_MUTEX_DEFINE(data_mutex)`).

- Cuando llega un dato de visión (`update_bbox`), el sistema bloquea el mutex, escribe las coordenadas compartidas (`shared_objX`) y libera el cerrojo.
- Cuando el hilo de lógica necesita calcular el control proporcional, solicita el mutex. Si está ocupado, el RTOS pone al hilo en espera hasta que los datos sean consistentes, garantizando una operación atómica.

4.4. Implementación de Alto Nivel (Subsistema Linux)

Mientras el STM32 se encarga de la supervivencia física del robot, el procesador Qualcomm ejecuta la lógica de aplicación en Python. El software desarrollado se divide en dos módulos principales: el orquestador principal (`main.py`) y el cliente de comunicaciones (`iot_client.py`).

4.4.1. Orquestación y Concurrencia (`main.py`)

El desafío principal en el lado de Linux es la gestión del recurso compartido **Arduino Bridge**. Tanto el subsistema de visión (que quiere enviar coordenadas) como el de telemetría (que quiere pedir el voltaje de la batería y los datos de velocidad y dirección) necesitan usar el mismo canal RPC. Si ambos intentan escribir simultáneamente sin control, el canal se satura o corrompe.

Para resolver esto, se implementó un modelo de **Acceso Prioritario con Mutex** (`threading.Lock`).

El reconocimiento de objetos se instrumenta mediante el componente modular **Brick** integrado en Arduino App Lab, concretamente a través de la clase `VideoObjectDetection`. Esta librería abstrae la complejidad del *pipeline* de visión computacional, exponiendo una API de alto nivel que aprovecha la aceleración hardware del procesador para la inferencia de modelos.

Para integrar este flujo continuo de imágenes con la lógica de control, se implementó un diseño reactivo basado en eventos: se registró la función `notify_microcontroller` como un *callback* asíncrono mediante el método `on_detect.all`. De este modo, el motor de visión invoca automáticamente a esta rutina únicamente cuando identifica presencias positivas, inyectando las coordenadas y el nivel de confianza de la detección directamente en el flujo de ejecución principal.

- **Pre-procesamiento:** Se filtran las detecciones con confianza menor al 40 % y se calcula el centroide (X, Y) del cuadro delimitador (*bounding box*).
- **Sección Crítica:** El hilo de visión solicita el *lock* del Bridge de forma bloqueante (`with bridge_lock`). Esto garantiza que, si hay una detección, el sistema esperará lo necesario para enviarla, ya que la navegación es la prioridad absoluta.

Para evitar que la lectura de la batería o la velocidad retarde el bucle de visión, el hilo de telemetría implementa una política de "Mejor Esfuerzo" (*Best Effort*). Utiliza la llamada `acquire(blocking=False)`. Si el canal RPC está ocupado por la visión, la telemetría **desiste** en ese ciclo y espera al siguiente, en lugar de bloquear el sistema.

```

1 # Hilo de Telemetria (Baja Prioridad)
2 def telemetry_poller(iot_client):
3     while True:
4         # Intenta coger el candado. Si esta ocupado por Vision, devuelve False.
5         is_acquired = bridge_lock.acquire(blocking=False)
6
7         if is_acquired:
8             try:
9                 # Canal libre: Pedimos datos al MCU
10                response = Bridge.call("get_telemetry")
11                if response:
12                    iot_client.update_data(json.loads(response))
13            finally:
14                bridge_lock.release()
15        else:
16            # Si el Bridge esta ocupado, saltamos este ciclo
17            # Priorizamos la latencia de la vision sobre el log de bateria
18            pass
19
20    time.sleep(0.2)

```

Listing 7: Implementación de QoS en main.py: Telemetría no bloqueante

4.4.2. Pasarela IoT Desacoplada (`iot_client.py`)

Para la comunicación con el exterior se desarrolló una clase `IoTClient` que encapsula la librería **Paho MQTT**. El diseño implementa el patrón *Producer-Consumer* para desacoplar la adquisición de datos (que ocurre en `main.py` vía RPC) del envío de red.

Se han tomado tres decisiones de diseño clave para garantizar el rendimiento:

1. **Thread-Safety Interno:** La clase posee su propio `threading.Lock` para proteger el diccionario `latest_data`. Esto permite que el hilo principal actualice los datos a alta velocidad (200ms) sin riesgo de colisión de memoria con el hilo de red.
2. **Loop de Publicación Independiente:** Se genera un hilo demonio (*daemon thread*) que publica el estado en el tópico MQTT `robot/telemetry` a una frecuencia fija de 1Hz. Esto estabiliza el consumo de ancho de banda, evitando inundar el Dashboard con datos redundantes si el bucle de control se acelera.
3. **Estrategia de Transporte (QoS 0):** Se configuró el cliente MQTT con Nivel de Calidad de Servicio 0 (*At most once*). En sistemas de telemetría de robots móviles, la frescura del

dato es más importante que la integridad histórica.

- Si se pierde un paquete de “Batería: 7.4V”, es inútil retransmitirlo un segundo después porque ya habrá llegado una nueva lectura más actual.
- El uso de QoS 0 elimina el *handshake* de confirmación (PUBACK) del protocolo TCP/IP, reduciendo la latencia de red y la carga de CPU del procesador.

```

1 def update_data(self, data_dict):
2     # Metodo seguro para actualizar datos desde el hilo principal
3     with self.lock:
4         self.latest_data = data_dict
5
6 def _publish_loop(self):
7     # Bucle de envio constante a 1Hz
8     while self.running:
9         with self.lock:
10            if self.latest_data:
11                # QoS=0 por defecto: Fire-and-forget para menor latencia
12                self.client.publish(self.topic, json.dumps(self.latest_data),
13                qos=0)
14            time.sleep(1.0)

```

Listing 8: Desacoplamiento de red y publicación QoS 0 en `iot_client.py`

5. Montaje Físico

El desarrollo del hardware se ha regido por una filosofía de *upcycling* tecnológico. En lugar de fabricar un chasis desde cero, se tomó como base mecánica el kit educativo Elegoo Smart Robot Car V3.0.

Esta decisión permitió centrar los esfuerzos en la arquitectura de software y control, aprovechando una plataforma mecánica robusta y probada. El proyecto transforma este juguete teledirigido en un robot autónomo avanzado mediante la sustitución de su cerebro original (un Arduino UNO R3 estándar de 8 bits) por la nueva Arduino UNO Q, ampliando drásticamente sus capacidades de cómputo sin alterar su huella física.

5.1. Compatibilidad del Ecosistema y Shield de Expansión

Para la gestión de potencia se reutilizó la placa de expansión (*Hat*) original del kit: el Elegoo Smart Car Shield V3.0. La gran ventaja de la **Arduino UNO Q** es que, pese a su arquitectura interna heterogénea (MPU Qualcomm + MCU STM32), respeta estrictamente el factor de forma y el *pinout* eléctrico del estándar Arduino UNO. Esto permitió acoplar el Shield directamente sobre la nueva placa base sin adaptadores, heredando instantáneamente la gestión de los motores y servos.

5.2. Integración Híbrida e Instrumentación Sensorial

El diseño electrónico final es una solución híbrida que combina la electrónica de consumo del kit base con instrumentación de grado superior añadida ad-hoc:

- **Electrónica Base (Reutilizada):** Se emplea la etapa de potencia del Shield (doble puente H L298N) para la tracción y los reguladores de voltaje para los servos. Se descartaron módulos obsoletos del kit original, como el receptor Bluetooth HC-05 o los sensores de línea básicos, cuyas funciones son ahora asumidas por el stack de conectividad y visión de la UNO Q.

- **Instrumentación de Odometría (Encoders Magnéticos):** Dado que el kit original opera en lazo abierto (sin retroalimentación de velocidad), fue necesario instrumentar el chasis para permitir el control proporcional. Se integraron encoders de cuadratura basados en efecto Hall:
 - Se adhirieron discos magnéticos multipolares a las ruedas del robot.
 - Se instalaron sensores Hall KY-003 en el chasis, alineados con los imanes, para detectar la rotación mediante la variación del campo magnético.
 - Estos sensores se conectaron a las interrupciones externas (Pines 2 y 10) de la UNO Q, permitiendo al MCU contar pulsos de movimiento con precisión milimétrica sin saturar al procesador principal.
- **Pines Expuestos (Breakout):** Se aprovecharon las cabeceras pasantes de la placa para conectar el bus I2C (SDA/SCL) al giroscopio MPU6050 y al monitor de batería INA219, elementos inexistentes en el diseño original de Elegoo.

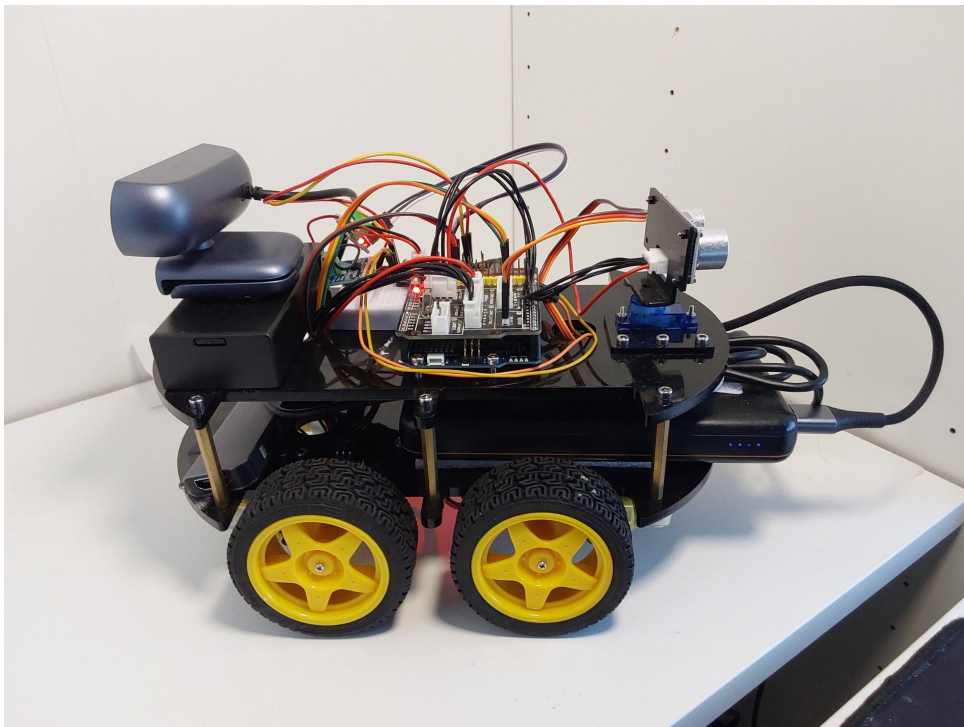


Figura 2: Detalle del montaje final: La Arduino UNO Q (abajo) sustituye al controlador original, gestionando la lógica avanzada, mientras el Shield (arriba) distribuye la potencia.

5.3. Arquitectura de Alimentación y Limitaciones del USB-C

Un desafío crítico encontrado durante la integración del hardware fue la gestión de la energía para el subsistema de visión.

El diseño original contemplaba una única fuente de energía (batería Li-Ion 2S conectada al pin *Vin*) para alimentar tanto los motores como la lógica de la placa. Sin embargo, al conectar la cámara web al puerto USB-C de la Arduino UNO Q, el periférico no recibía alimentación.

Tras analizar el esquema de distribución de energía (*Power Tree*) del fabricante, se identificó que la topología de la placa presenta una restricción en el modo *Host*:

- El regulador interno (*Step-Down*) convierte el voltaje de entrada V_{in} (7-12V) a 5V para la lógica interna (STM32 y Qualcomm).
- Sin embargo, este carril de 5V internos **no se inyecta de vuelta** hacia el conector USB-C con la potencia necesaria para levantar un periférico de alto consumo como una cámara web. El puerto USB-C está diseñado principalmente como entrada de energía (*Sink*) o datos, pero carece de la etapa de potencia necesaria para actuar como fuente (*Source*) robusta cuando el sistema se alimenta exclusivamente por V_{in} .

Para solventar esta limitación de hardware sin poner en riesgo los reguladores internos de la placa, se optó por separar los dominios de alimentación:

1. **Dominio de Tracción y Lógica (V_{in}):** La batería principal alimenta el Shield L298N (motores) y el pin V_{in} de la Arduino (procesadores y sensores I2C).
2. **Dominio de Visión (USB Dock):** Se integró un *Dock* o batería externa conectada al puerto USB-C. Esta fuente secundaria se encarga exclusivamente de negociar la conexión USB y suministrar los 500mA estables que requiere la cámara.

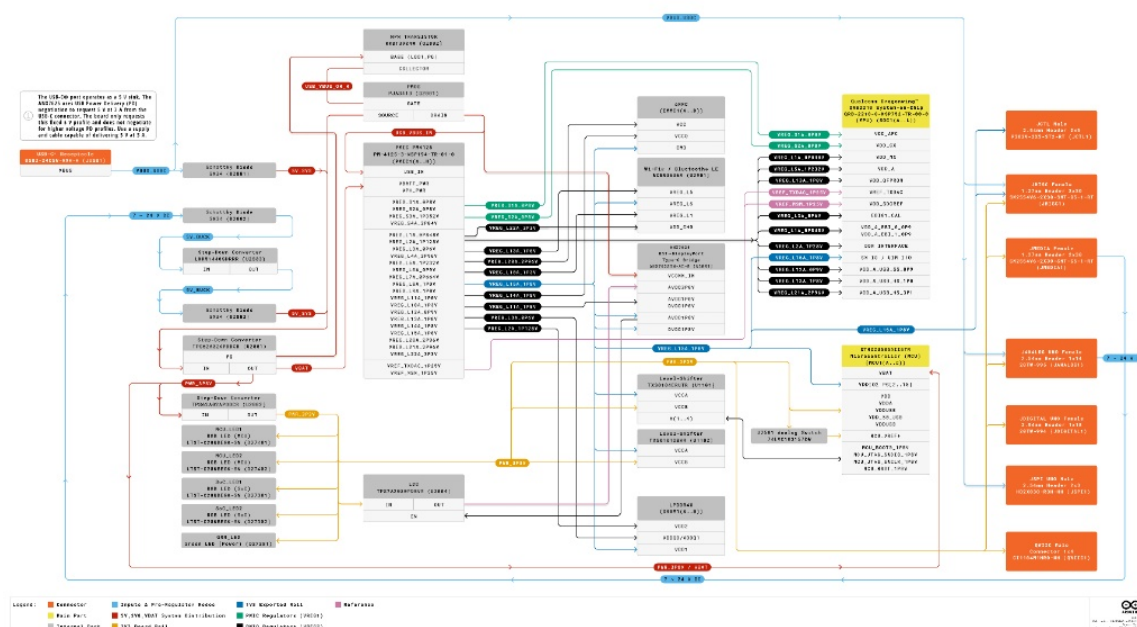


Figura 3: Diagrama del Árbol de Potencia (Power Tree). Se observa la complejidad de la gestión de energía y la separación de las líneas de alimentación, justificando el uso de una fuente externa para el bus USB.

6. Resultados y Validación Experimental

Para cuantificar el desempeño de la arquitectura híbrida propuesta, se sometió al prototipo a una serie de pruebas de estrés en entorno real. A continuación se presentan los resultados obtenidos respecto al consumo de recursos, la estabilidad de la red y el comportamiento de los sensores.

6.1. Análisis de Recursos Computacionales (CPU/RAM)

Se monitorizó el estado del microprocesador Qualcomm QRB2210 durante una sesión de seguimiento activo (ejecución simultánea de: captura de video, inferencia de red neuronal, lógica de control Python y cliente MQTT). La herramienta de diagnóstico `htop` arrojó los siguientes datos:

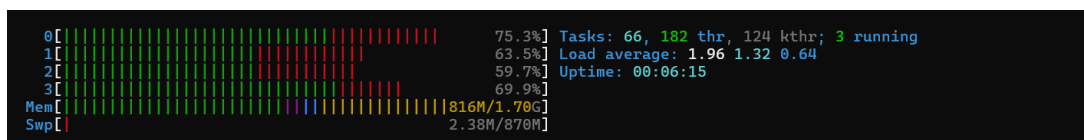


Figura 4: Monitorización de recursos en tiempo real. Se observa un balanceo de carga eficiente entre los 4 núcleos durante la inferencia de IA.

- **Gestión Multicúcleo:** A diferencia de las arquitecturas monolíticas donde el proceso de Python suele saturar un solo núcleo al 100 % (bloqueo por GIL), se observa una distribución de carga equilibrada entre los 4 núcleos Cortex-A53 (rango operativo del 60 % al 75 %). Esto demuestra que el sistema operativo Zephyr y los drivers de bajo nivel están gestionando eficazmente las interrupciones, evitando cuellos de botella térmicos.
- **Carga del Sistema (Load Average):** El promedio de carga de 1 minuto se situó en **1.96**. Dado que se trata de un procesador de cuatro núcleos, este valor indica que el sistema está operando aproximadamente al 50 % de su capacidad máxima de procesamiento, dejando margen suficiente para futuros procesos computacionales más complejos (como SLAM o reconocimiento de voz).
- **Consumo de Memoria:** El uso de memoria RAM se estabilizó en **816 MB** (aprox. 47 % del total disponible de 2GB). Esto valida la elección de la versión de 2GB de la Arduino UNO Q, confirmando que es capaz de sostener el stack de visión artificial y el sistema operativo Debian sin recurrir al intercambio de memoria en disco (*Swap*), lo que degradaría el rendimiento.

6.2. Latencia y Telemetría IoT

Se evaluó el rendimiento del sistema de comunicación distribuido (MQTT sobre Wi-Fi).

- **Estabilidad del Dashboard:** La interfaz web mostró una tasa de refresco constante de 1Hz. La decisión de configurar el cliente con QoS 0 permitió una transmisión fluida de la velocidad y el voltaje; las pérdidas de paquetes esporádicas resultaron imperceptibles para el usuario final gracias a la alta frecuencia de actualización.

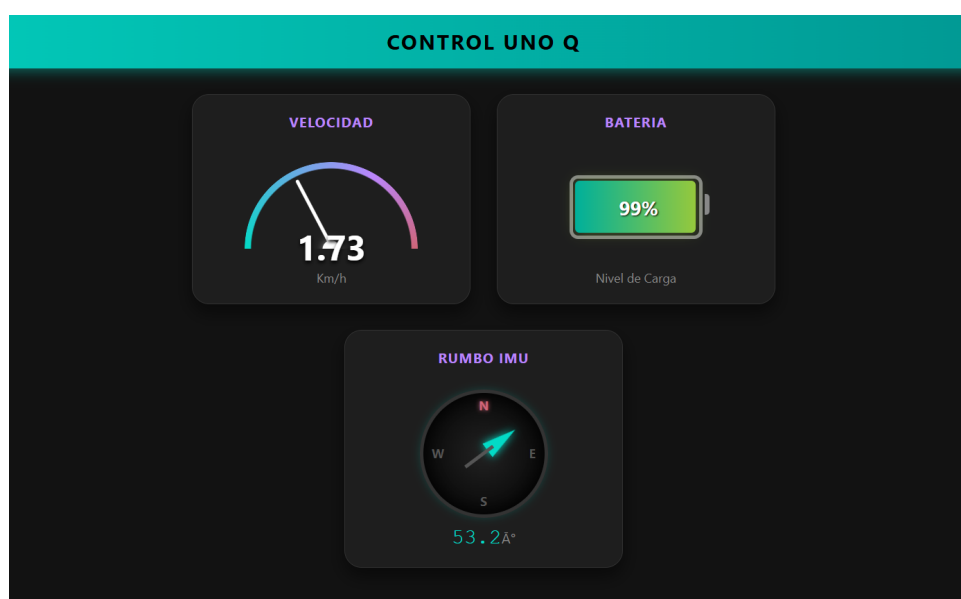


Figura 5: Dashboard de control del robot

- **Latencia de Control:** Mediante observación empírica, el retraso entre la detección visual (monitorizada por SSH) y la reacción de los motores se estimó en un intervalo inferior a un segundo, lo cual es suficiente para el seguimiento de personas a velocidad de paso humano ($\approx 1m/s$).

6.3. Pruebas de Campo: Visión y Evasión

Se realizaron pruebas físicas en un entorno de campo con iluminación natural.

1. **Rango de Detección:** El algoritmo de visión mantuvo un seguimiento fiable del rostro humano hasta una distancia máxima de 3.6 metros. Más allá de este umbral, la resolución del sensor de imagen limitaba la capacidad de la NPU para inferir rasgos faciales con una confianza superior al 40 %.
2. **Comportamiento de Evasión:** La máquina de estados de evasión (*State Avoiding Sequence*) demostró una tasa de éxito del 100 % en la detección de objetos a 20cm. La pausa de estabilización de 200ms implementada en el firmware eliminó eficazmente los falsos positivos generados por las vibraciones del chasis, permitiendo al robot navegar en entornos confinados sin colisiones.

7. Conclusiones y Trabajo Futuro

7.1. Conclusiones Generales

El desarrollo y validación de este prototipo ha demostrado la viabilidad y robustez de utilizar una arquitectura de computación asimétrica (Linux + Zephyr RTOS) para la robótica móvil de bajo coste. Tras el análisis experimental, se extraen las siguientes conclusiones principales:

1. **Eficacia del Desacoplamiento Híbrido:** Se ha validado que es posible (y deseable) separar la carga cognitiva de la reactiva. Delegar la visión artificial al procesador de aplicación y el control de motores al microcontrolador en tiempo real elimina el *jitter* en la navegación, permitiendo tiempos de reacción deterministas ($< 10ms$) independientemente de la carga de la red neuronal.
2. **Importancia del Software de Bajo Nivel:** La fluidez del robot no depende tanto del algoritmo de visión, sino de la calidad de los drivers intermedios. La implementación de zonas muertas por software, rampas de aceleración y filtrado de señales en los encoders ha resultado ser crítica para convertir decisiones digitales discretas en movimientos físicos suaves.
3. **Ventajas del RTOS en Sensores Críticos:** La gestión del giroscopio MPU6050 mediante un hilo de alta prioridad en Zephyr ha solucionado los problemas de deriva por integración temporal inestable, demostrando que un sistema operativo de tiempo real es superior al modelo de "Super-Loop" de Arduino clásico para tareas de navegación inercial.
4. **Eficiencia de Costes y Accesibilidad:** Uno de los hallazgos más relevantes del proyecto es la drástica reducción de costes lograda frente a las plataformas académicas estándar. Al utilizar componentes genéricos de mercado (motores TT, drivers L298N) gobernados por la Arduino UNO Q, se ha logrado un robot funcional por menos de 140€. Como se detalla en la Tabla 1, esta solución es un 44 % más económica que el Duckiebot oficial, un Robot con las mismas capacidades que el propuesto, eliminando la barrera económica para la enseñanza de IA avanzada en el aula.

Tabla 1: Desglose de costes reales frente al estándar de industria.

Componente (Materiales Individuales)	Precio Mercado (€)
Arduino UNO Q	47.58
4x Motores DC TT + Ruedas + Chasis Acrílico	18.50
Driver de Potencia L298N	2.65
Cámara Web USB (Genérica HD 720p/1080p)	22.65
Sensores (MPU6050 + 2x KY-003 + INA219)	14.50
Sistema Energía (2x18650 Li-Ion + Holder + PowerBank)	30.00
Total Prototipo Desarrollado	135.88 €
<i>NVIDIA Duckiebot</i>	310,52 €

7.2. Trabajo Futuro y Escalabilidad

El prototipo actual constituye una prueba de concepto funcional (PoC). Sin embargo, disponiendo de mayor tiempo de desarrollo, presupuesto y un equipo de ingeniería ampliado, el proyecto podría evolucionar en las siguientes direcciones estratégicas:

7.2.1. 1. Unificación del Sistema de Energía (Hardware)

Actualmente, el sistema depende de una topología de energía dual (Batería Li-Ion para motores + Power Bank USB para la cámara) debido a las limitaciones de corriente del puerto USB-C de la placa en modo *Host*.

- **Solución Propuesta:** Diseñar una PCB de regulación intermedia que tome los 7.4V de la batería principal y entregue 5V/3A estables directamente a las líneas VBUS del puerto USB-C. Esto permitiría alimentar la cámara y la placa con una **única batería**, reduciendo el peso total del robot y simplificando la operación de carga.

7.2.2. 3. Fusión Sensorial Robusta (EKF)

Para mejorar la precisión de la odometría en superficies deslizantes, se propone implementar un **Filtro de Kalman Extendido (EKF)**.

- Este algoritmo fusionaría matemáticamente los datos de los encoders (precisos a corto plazo) con los del giroscopio (precisos en giros) y la visión (corrección absoluta), eliminando el error acumulativo característico de la navegación a estima.

7.2.3. 4. Optimización de la Electrónica de Potencia

El uso del driver L298N (tecnología bipolar antigua) implica una caída de tensión de casi 2V que se disipa en calor, reduciendo la eficiencia.

- **Mejora:** Migrar a drivers basados en tecnología MOSFET (como el TB6612FNG o diseños propios). Esto incrementaría el par motor disponible y extendería la autonomía de la batería entre un 15 % y un 20 %.



Referencias

- [1] “Github - nvidia-ai-iot/jetbot: An educational ai robot based on nvidia jetson nano..”
- [2] “Uno q — arduino documentation.”
- [3] “Threads — zephyr project documentation.”
- [4] “Uno q user manual — arduino documentation.”
- [5] P. B. ALLDATASHEETCOM, “L298n dual full-bridge driver,” tech. rep., 2000.
- [6] “Mpu-6000 and mpu-6050 product specification revision 3.4 mpu-6000/mpu-6050 product specification,” tech. rep., 2013.
- [7] “Robot. head to toe product user’s manual-hcsr04 ultrasonic sensor user’s manual,” tech. rep., 2013.
- [8] P. B. ALLDATASHEETCOM, “Acs715 allegro — alldatasheet,” tech. rep.
- [9] “Ina219,” tech. rep., 2008.
- [10] “Sen-ky003-datasheet,”
- [11] “Elegoo ir remote controller,”